NVFP4 Pretraining Recipe

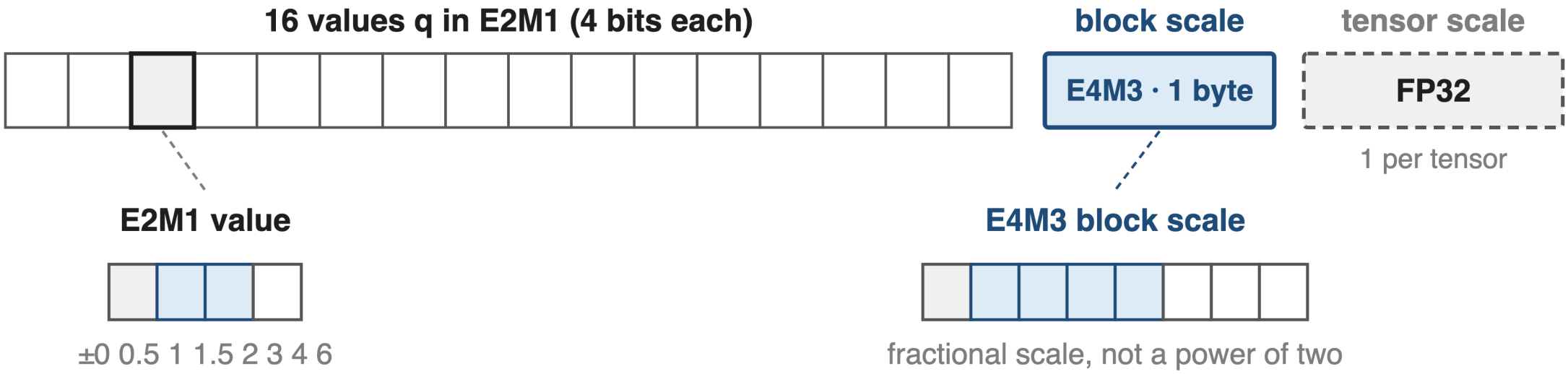
4-bit GEMMs for LLM pretraining at near-FP8 loss

NVIDIA · arXiv 2509.25149 · Transformer Engine · Megatron-Bridge

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Low-Precision & Quantization · arxiv.org/abs/2509.25149

NVFP4: 16 four-bit values share an FP8 scale



$$x = q \times s_{block} \times s_{tensor}$$

MLPerf Training: Llama 3.1 405B pretrained in NVFP4 on 512 GB300 GPUs in 64.6 minutes

Naive 4-bit formats waste tokens or diverge

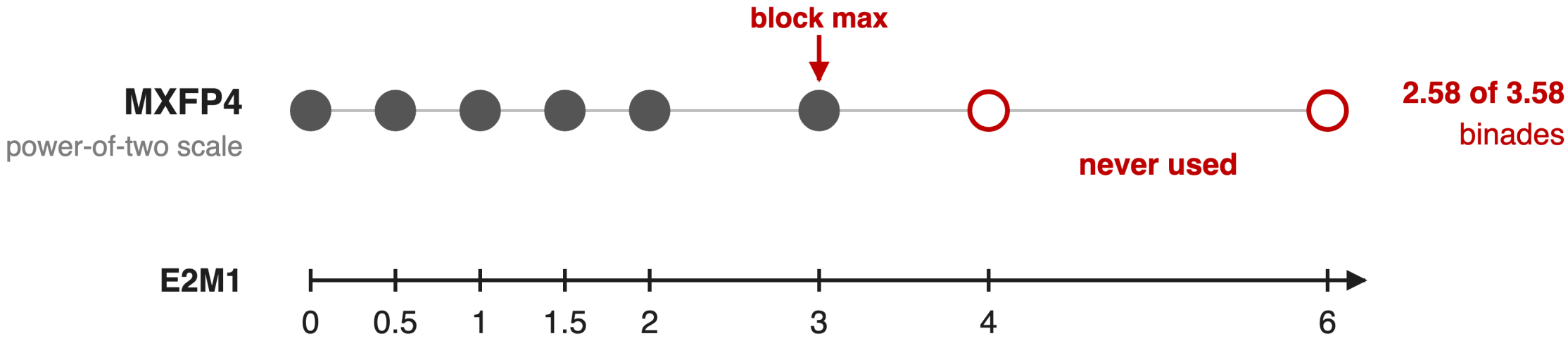
+36%

more training tokens MXFP4 needs to reach NVFP4's loss (8B model, 1T tokens)

- All layers in NVFP4 with nearest rounding: **diverges early**

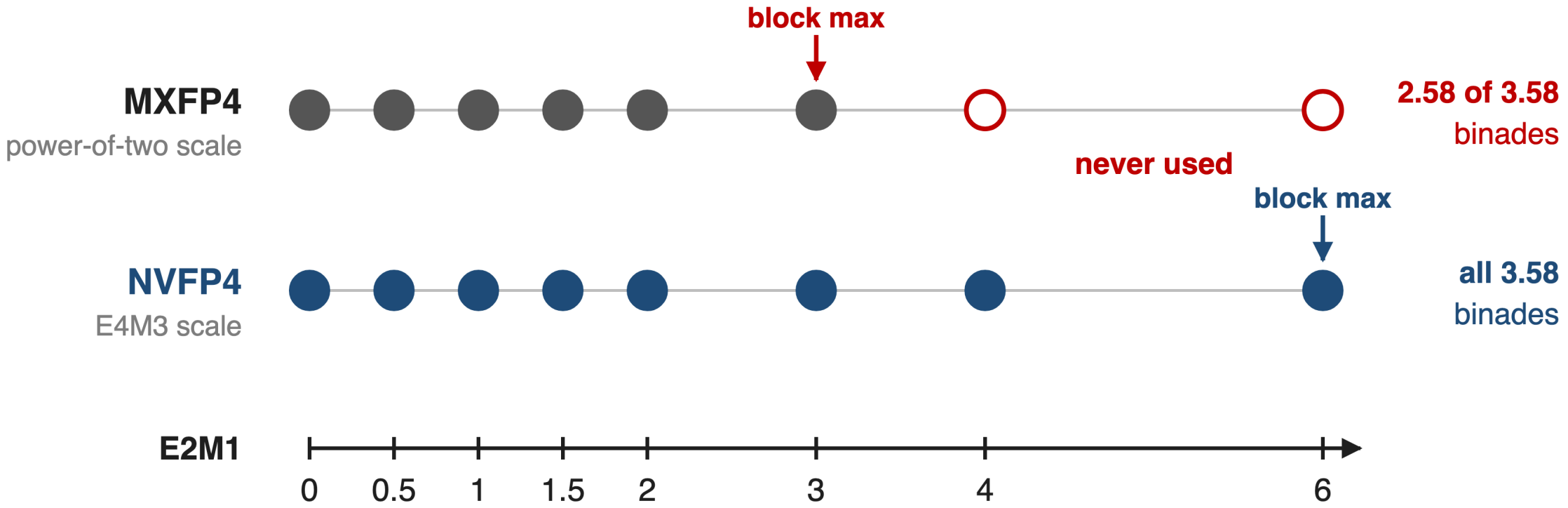
NVIDIA, "Pretraining LLMs with NVFP4", arXiv 2509.25149, Sec. 5 and Fig. 6 (same recipe for both formats)

Power-of-two scales waste FP4's top samples



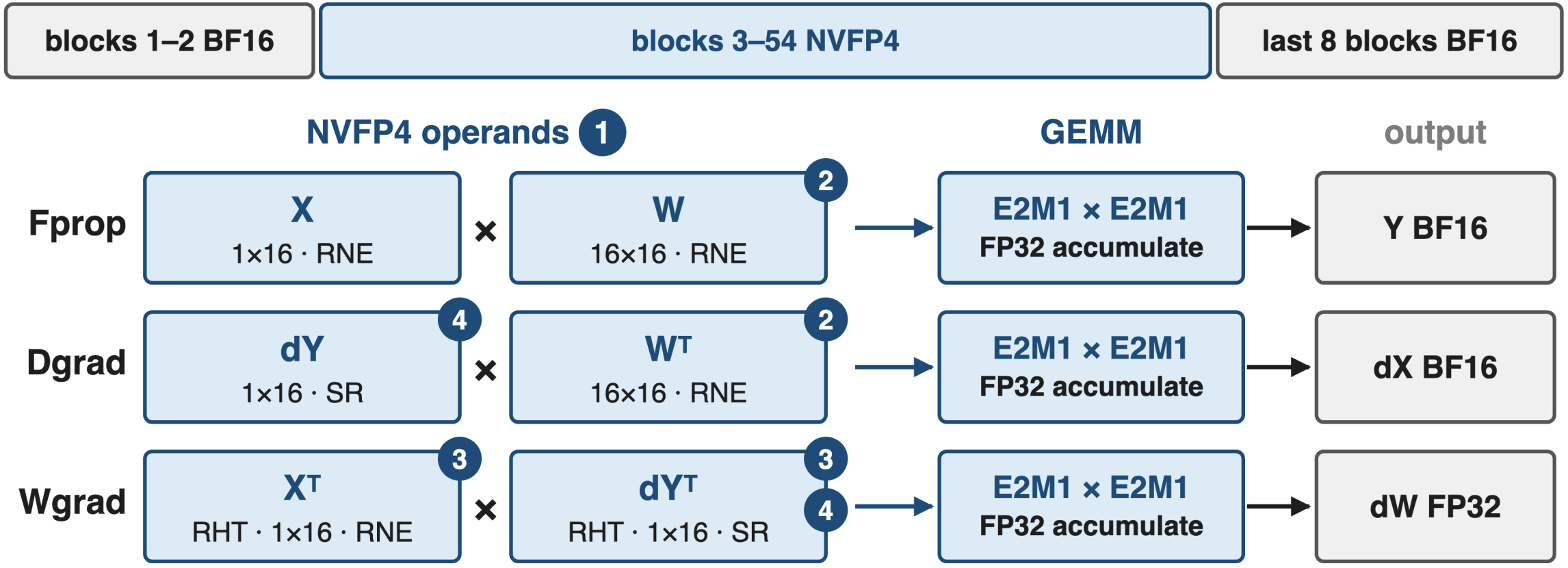
MXFP4 rounds its E8M0 scale up; worst case the block max lands at 3 and ±4, ±6 go unused

Insight: spend bits on the scale, not the values



An E4M3 scale per 16 values stores each block's max at near-FP8 precision, ≥ 6.25% of values

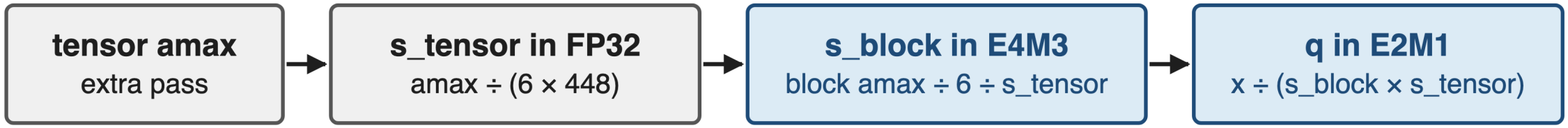
The recipe pairs one fix with each error source



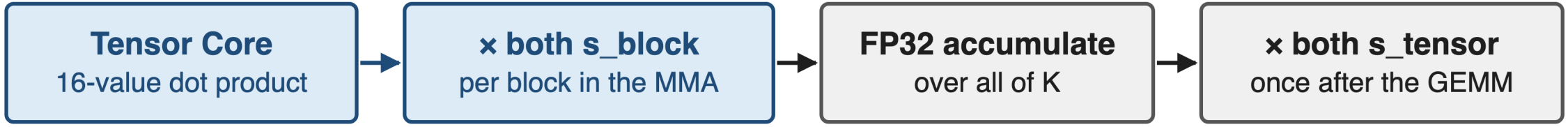
❶ two-level scales · ❷ 16×16 weight blocks · ❸ Hadamard on Wgrad inputs · ❹ stochastic rounding on gradients

❶ Tensor scale keeps block scales in E4M3 range

encode (quantize)



decode (inside the GEMM)

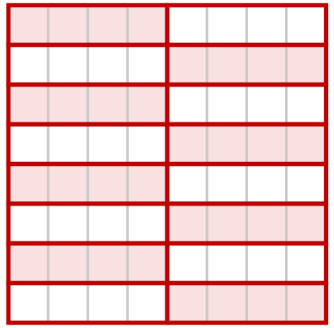


Block scales apply per 16 values inside the MMA; tensor scales once after the GEMM

❷ 16×16 weight blocks: one W in both passes

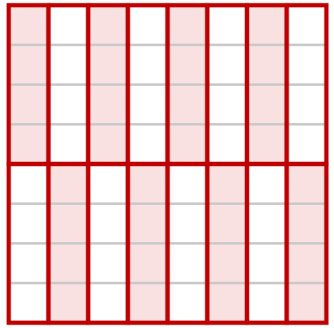
1×16 weight scales

W in Fprop



≠

W in Dgrad

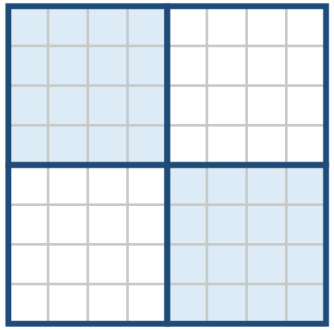


two different quantized W

backward differentiates another W

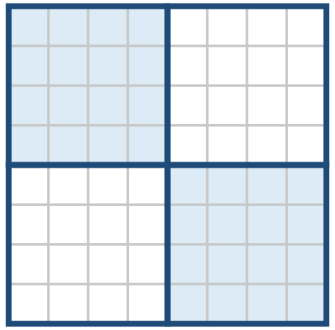
16×16 weight scales

W in Fprop



=

W in Dgrad

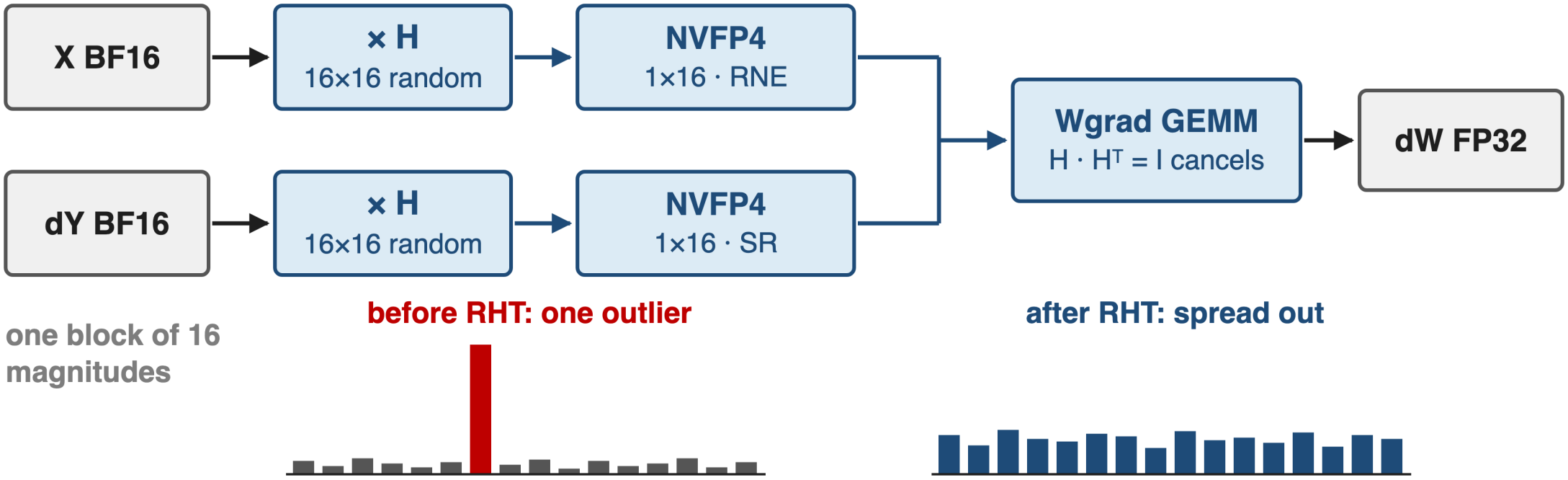


one quantized W

activations and gradients keep 1×16

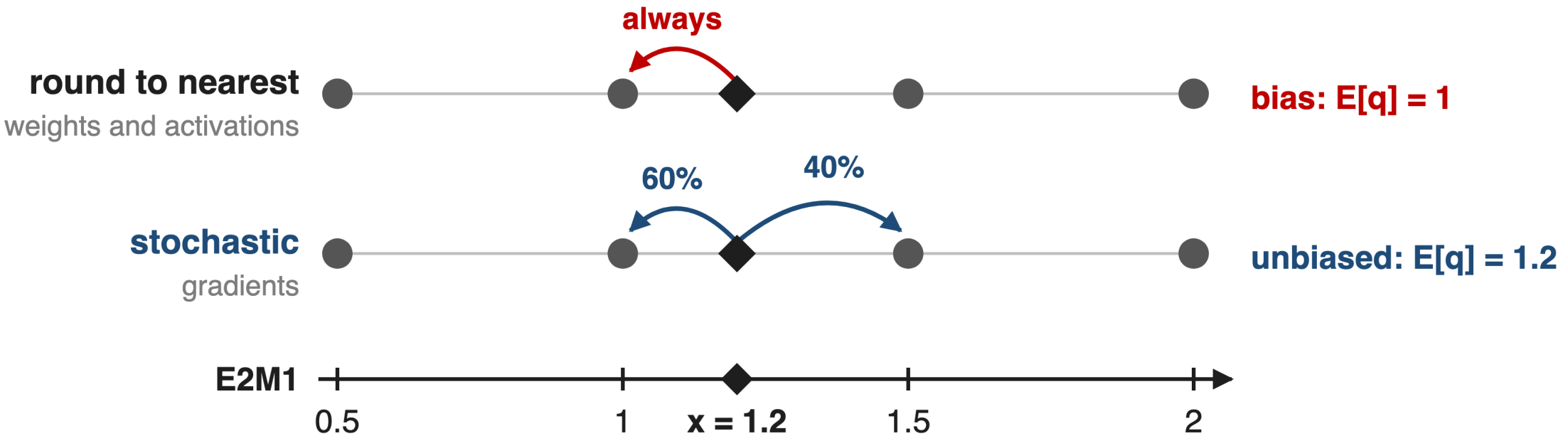
Weights adapt to coarser 16×16 blocks; activations and gradients keep finer 1×16 scales

❸ Rotate Wgrad inputs to spread outliers



16×16 Hadamard: better than 4×4, as good as 128×128 at 12B; only on Wgrad inputs

❹ Round gradients stochastically to remove bias



SR on gradients is essential at 12B; SR on weights or activations hurts

NVFP4 stays within 1.5% of FP8 loss at 10T

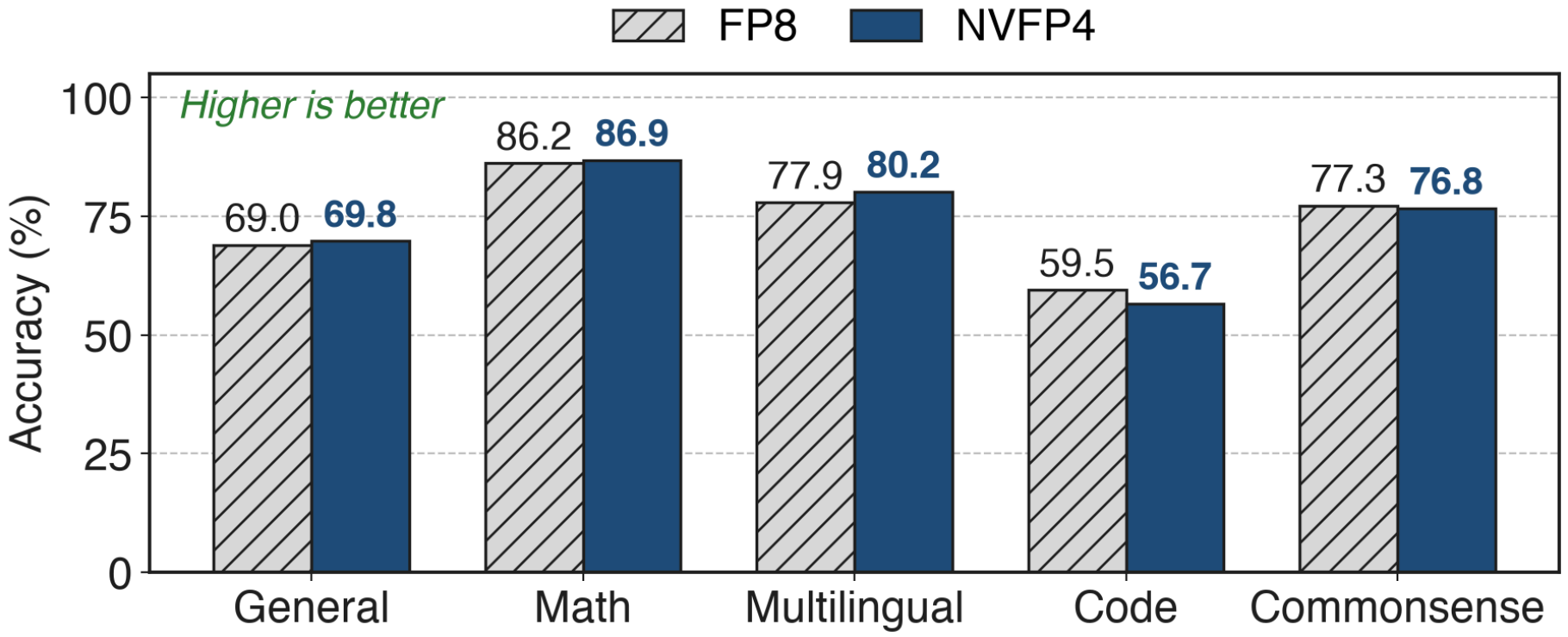
< 1%

relative loss gap vs FP8 in the stable phase (12B, 10T tokens); slightly above 1.5% after LR decay

- Longest public 4-bit pretraining run at the time: **12B** model, **10T** tokens

NVIDIA, arXiv 2509.25149, Sec. 3 and Fig. 2 (FP8 baseline: blockwise 1×128 / 128×128)

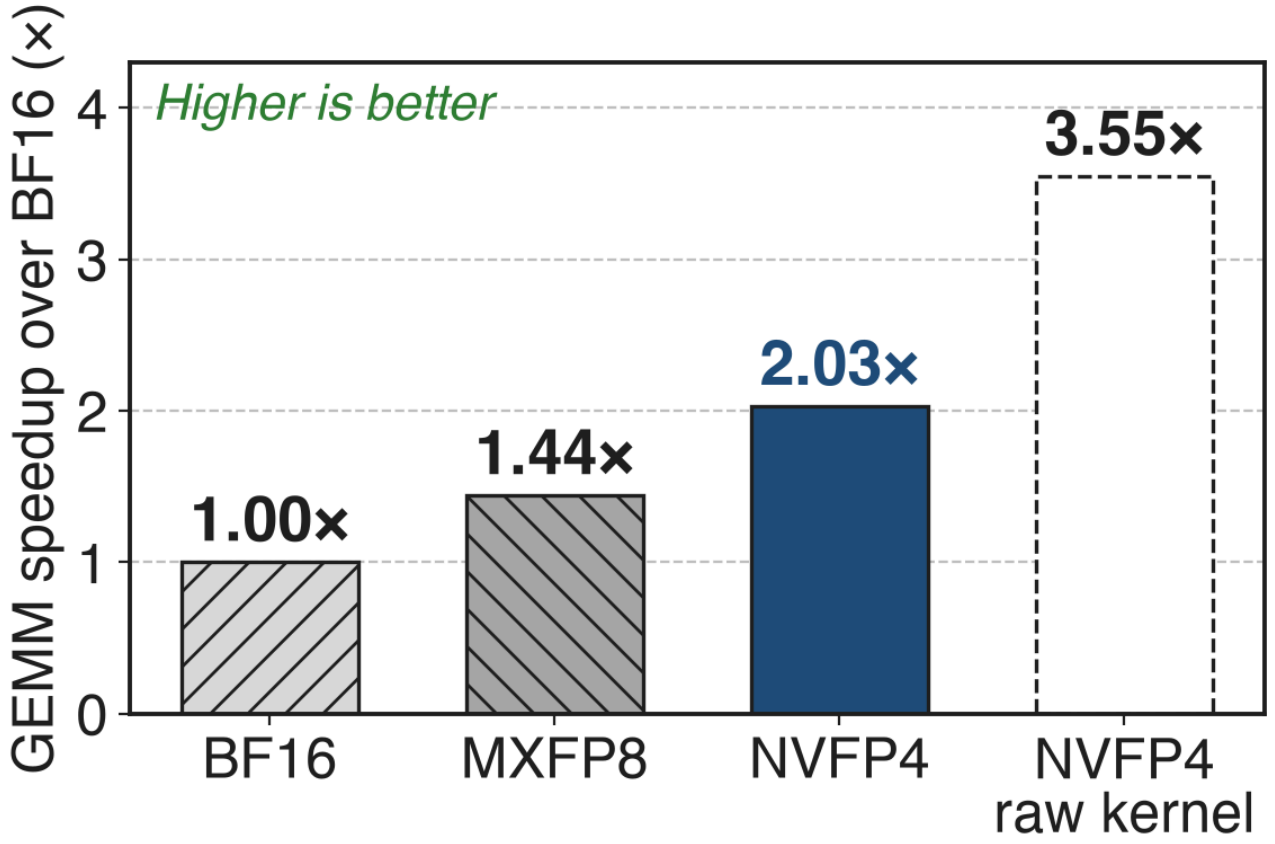
Accuracy matches FP8: MMLU-Pro 62.58 vs 62.62



Code trails by 2.9 points (MBPP+ 55.91 vs 59.11); the paper suspects noisy final-checkpoint evals

NVFP4 GEMMs run 2.03× BF16 on B300

- 5B model, quantization included
- **1.41×** over MXFP8 (derived)
- Raw kernel **3.55×**: quantize work eats the rest



The price: BF16 layers and a residual gap

- **16%** of linear layers stay BF16 at 12B
- **~1.5%** final loss gap; one 12B hybrid, no MoE yet
- Blackwell only; RHT needs BF16 inputs

Remedy: a BF16 forward pass from 8.2T tokens cuts the gap to 0.5% for ~6% of compute

Summary

- MXFP4 needs **+36%** tokens; naive NVFP4 diverges
- **Insight**: FP8 scale per 16 values, FP32 per tensor; then 16×16 weights, Wgrad Hadamard, SR on gradients
- **< 1%** (end **~1.5%**) loss gap to FP8 at 12B / 10T; **2.03×** BF16 GEMM speed on B300

https://arxiv.org/abs/2509.25149